The *receiver* is the object that you call a method on. For example, if you write my_string.reverse(), then my_string is the receiver.

To understand the concept of an *ancestors chain*, look at any Ruby class. Then imagine moving from the class into its superclass, then into the superclass's superclass, and so on, until you reach BasicObject, the root of the Ruby class hierarchy. The path of classes you just traversed is the ancestors chain of the class. (The ancestors chain also includes modules, but forget about them for now. We'll get around to modules in a bit.)

Now that you know what a receiver is and what an ancestors chain is, you can sum up the process of method lookup in a single sentence: to find a method, Ruby goes in the receiver's class, and from there it climbs the ancestors chain until it finds the method. Here's an example:

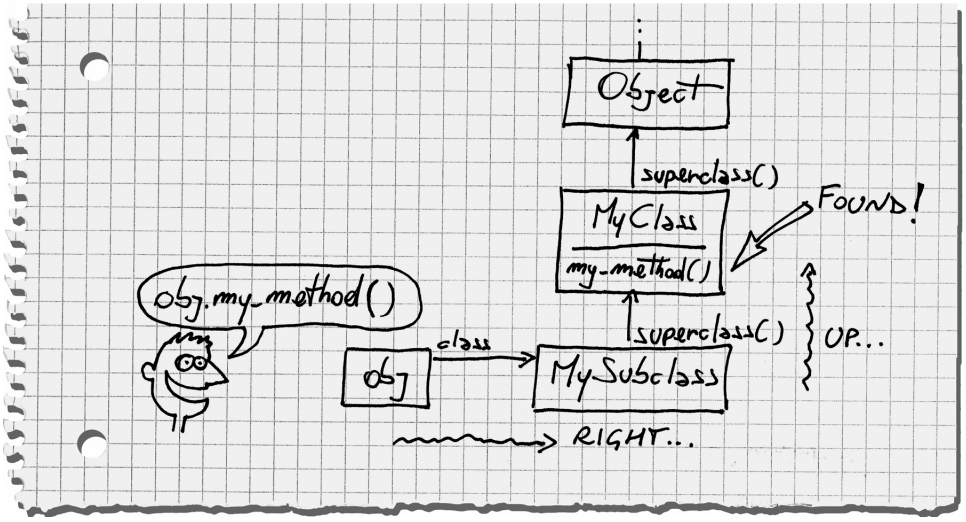
object_model/lookup.rb

```
class MyClass
  def my_method; 'my_method()'; end
end

class MySubclass < MyClass
end

obj = MySubclass.new
obj.my_method()       # => "my_method()"
```

Bill draws this diagram:



If you're used to traditional class diagrams, this picture might look confusing to you. Why is obj, a humble object, hanging around in the same diagram with